

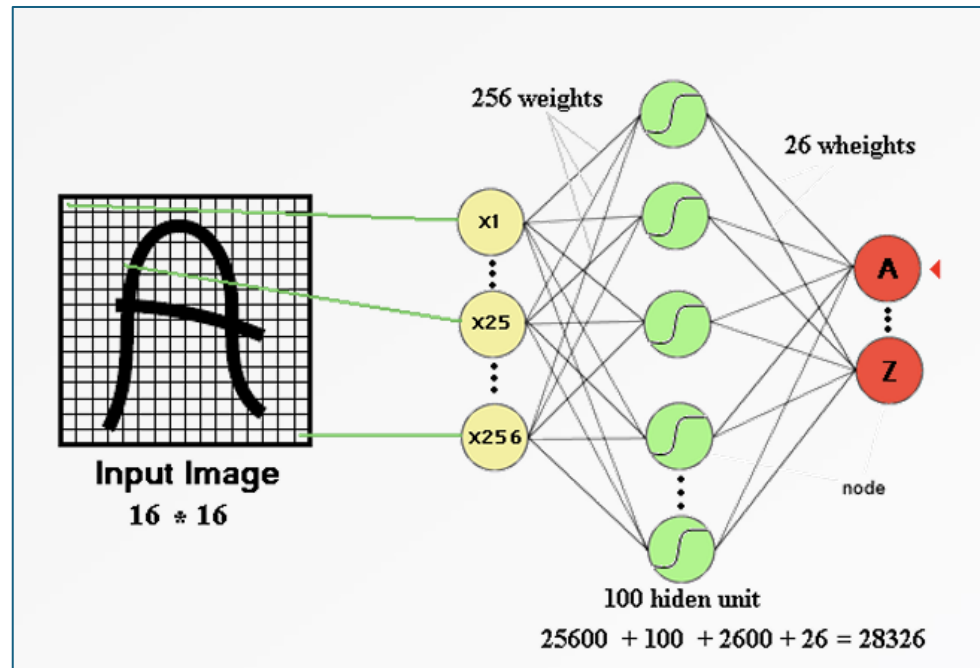
WORK



seperti MLP (Multi-Layer PERceptron) memiliki kelemahan jika langsung digunakan pada citra mentah. Hal ini terjadi oleh sebab berikut:

1. Jumlah Parameter Ekstrem

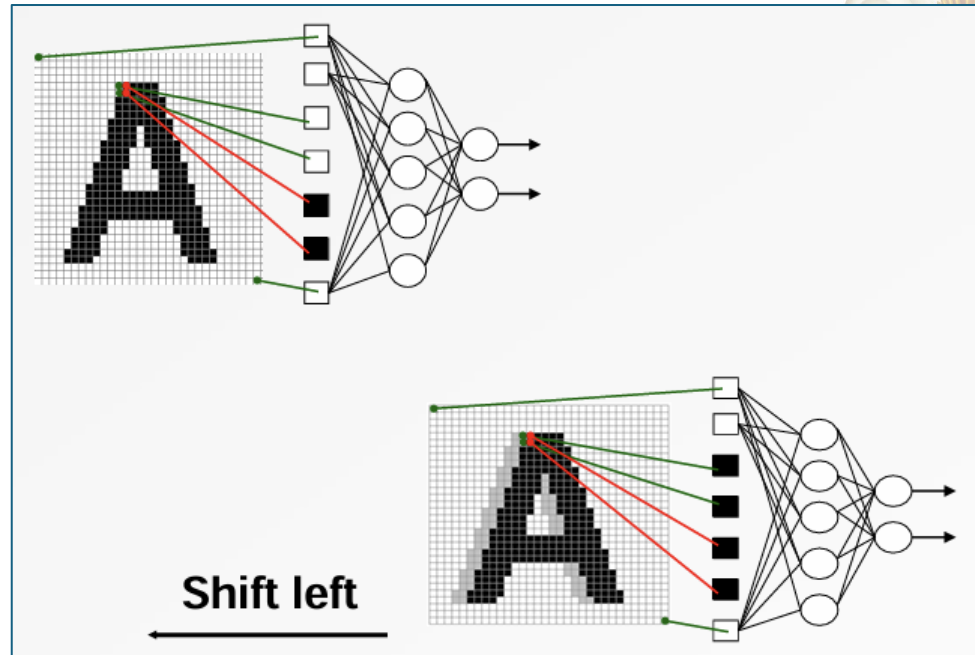
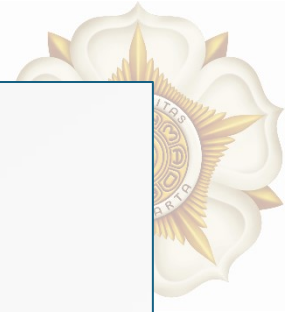
Citra kecil berukuran 16 x 16 saja jika dihubungkan ke hidden layer bisa menghasilkan puluhan ribu bobot (weights), yang menyulitkan komputasi.



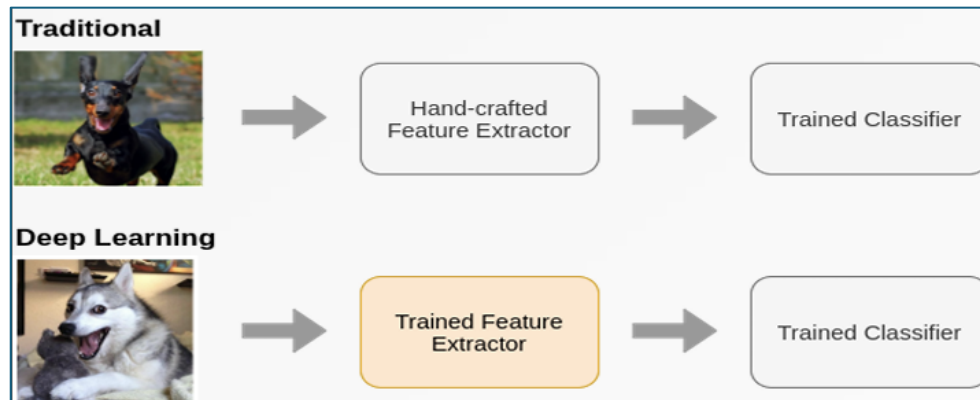
Gambar 0.2 Hidden Layer MLP

2. Hilangnya Invariansi Spasial

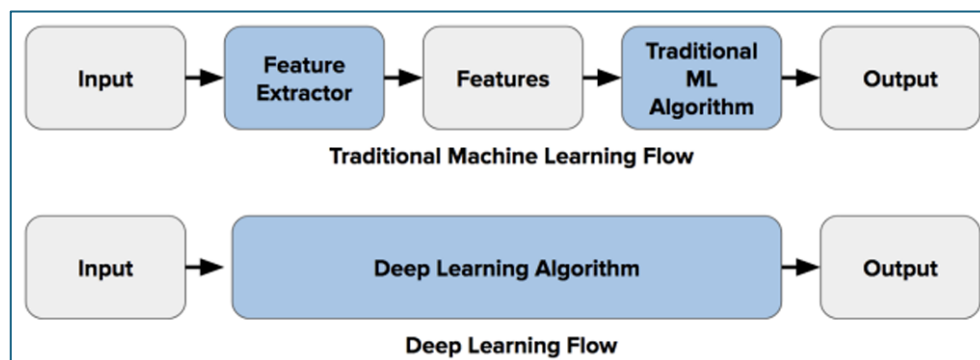
MLP tidak tahan terhadap pergeseran (shift), penskalaan (scaling), atau distorsi citra. Jika objek bergeser sedikit saja, MLP mungkin gagal mengenalinya.



Gambar 0.3 Pergeseran Citra pada MLP



Gambar 0.4 Ilustrasi Alur Tradisional vs Deep Learning



Gambar 0.5 Ilustrasi Alur Tradisional vs Deep Learning



- **Arsitektur Convolutional Neural Network (CNN)**

CNN adalah jaringan saraf *feed-forward* yang dirancang untuk mengekstrak pola visual langsung dari piksel dengan *preprocessing* minimal. Alur utama CNN terdiri dari:

1. Convolution Layer

Lapisan ini adalah inti dari CNN. Lapisan ini memiliki sejumlah Filter (kernel) yang “berjalan” di atas citra input untuk mengekstrak fitur. Operasi ini disebut konvolusi, yaitu perkalian titik (*dot product*) antara nilai filter dan bagian citra. Setiap filter akan menghasilkan *Feature Map* yang mendeteksi pola tertentu (misal: garis vertikal, lengkungan).

2. ReLU (Rectified Linear Unit)

Setelah konvolusi, fungsi aktivasi ReLU diterapkan untuk memperkenalkan non-linearitas. Rumusnya adalah $\Phi(x) = \max(x, 0)$, yang mengganti semua nilai negatif piksel menjadi nol.

3. Pooling Layer (Subsampling)

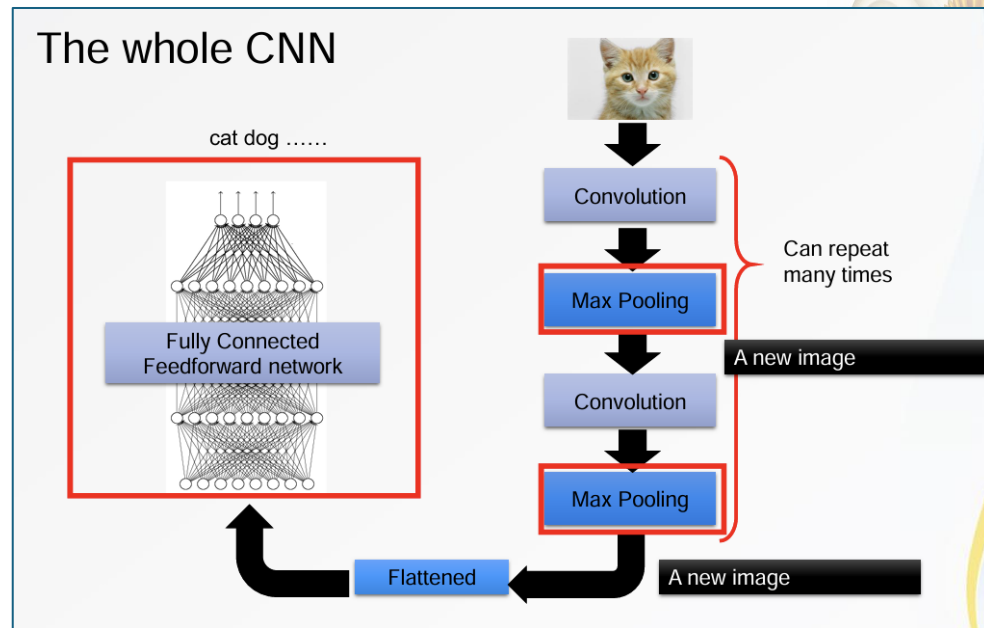
Lapisan ini berfungsi mengurangi dimensi citra (*downsampling*) agar parameter komputasi lebih sedikit dan model lebih kebal terhadap pergeseran objek 11. Teknik paling umum adalah Max Pooling, yang mengambil nilai terbesar dari area tertentu (misal 2 x 2).

4. Flattening

Mengubah matriks fitur multi-dimensi (hasil pooling terakhir) menjadi vektor satu dimensi (vektor baris panjang) agar bisa dimasukkan ke jaringan saraf biasa.

5. Fully Connected Layer (Dense).

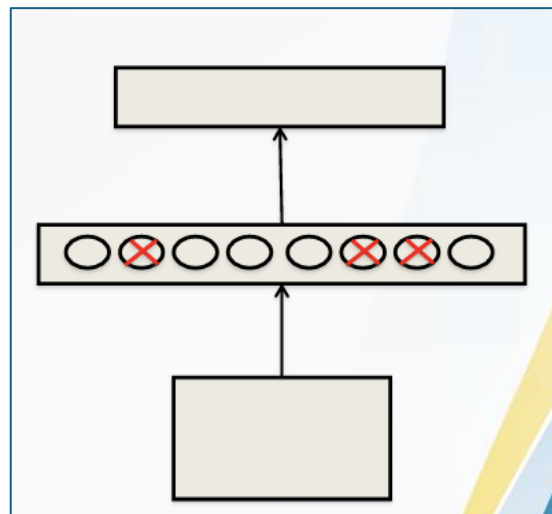
Bagian akhir dari CNN yang bertindak sebagai classifier (seperti MLP biasa). Lapisan ini menghubungkan semua fitur yang telah diekstrak ke kelas output (misalnya: Kucing, Anjing, Burung).



Gambar 0.6 Alur CNN secara keseluruhan

- **Drop Out**

Dropout adalah teknik regularisasi di mana neuron dimatikan secara acak (misalnya dengan probabilitas 0.5) selama proses pelatihan. Ini sangat efektif untuk mengurangi *overfitting* pada jaringan saraf yang dalam.



Gambar 0.7 Dropout

9.3. ALAT DAN BAHAN



- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Google Colab
- **Pustaka Python**
 - tensorflow
 - matplotlib

9.4. LANGKAH PERCOBAAN

Percobaan kali ini akan mencoba membuat model untuk klasifikasi gambar Dog vs Cat. Modul praktikum ini akan menggunakan sumber daya Google Colab untuk memanfaatkan GPU free tier. Berikut adalah langkah-langkah pembuatan model klasifikasi Dog vs Cat:

1. Unduh datasets

Unduh dataset Dog vs Cat dengan menuliskan perintah berikut di Google Colab. Jika sel berhasil dijalankan, folder dataset akan terbuat dan terlihat seperti Gambar 0.1.

```
!gdown --id "1hOU0OcYc8_za_0SJLc6HALjVc4sfVGNA" -O "datasets.zip"
```

```
!unzip -q datasets.zip
```

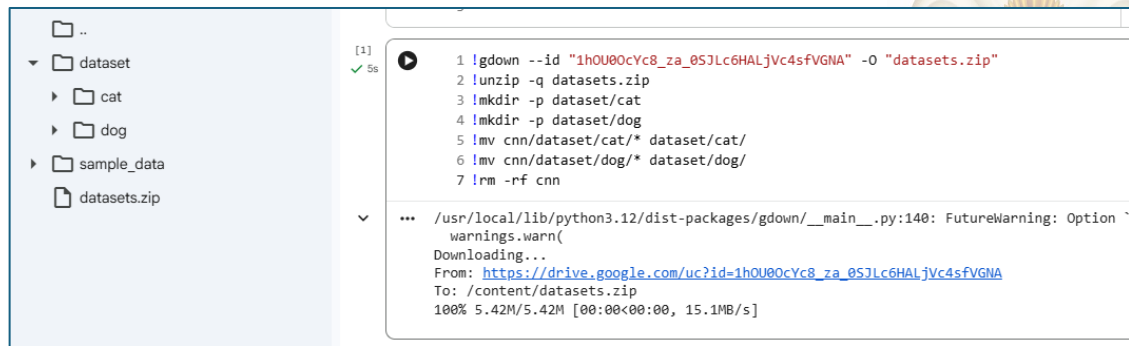
```
!mkdir -p dataset/cat
```

```
!mkdir -p dataset/dog
```

```
!mv cnn/dataset/cat/* dataset/cat/
```

```
!mv cnn/dataset/dog/* dataset/dog/
```

```
!rm -rf cnn
```



Gambar 0.1 Unduh Dataset Dog vs Cat

Jika Google Colab tidak bisa mengunduh dataset tersebut, gunakan link dibawah ini untuk mengunduh secara manual.

https://drive.google.com/file/d/1hOU0OcYc8_zs_0SJLc6HALjVc4sfVGNA

Setelah mengunduh secara manual, unggah file ke runtime agar terlihat seperti Gambar 0.1. Struktur folder harus terdiri dari dataset yang memiliki dua folder, yaitu cat dan dog. Setiap folder cat dan dog memiliki foto yang merepresentasikan folder tersebut.

2. Impor pustaka

Impor pustaka tensorflow, matplotlib, numpy, dan beberapa modul dari tensorflow keras. Secara default, Google Colab telah menyediakan pustaka tersebut, sehingga tidak perlu mengunduh pustaka. Tambahkan kode impor pustaka seperti Gambar 0.2.

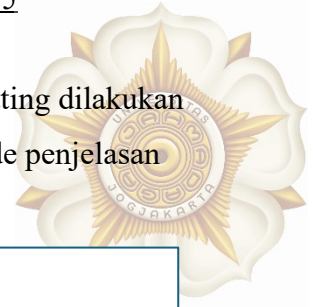
```

1 import os
2
3 import tensorflow as tf
4 import matplotlib.pyplot as plt
5 import os
6 import random
7 import numpy as np
8
9 from tensorflow.keras import Sequential
10 from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Dropout, Flatten
  
```

Gambar 0.2 Impor Pustaka

3. Pembacaan dan pembagian dataset

Setelah melakukan impor dan mengunduh dataset, tahap selanjutnya adalah membuat pembacaan dataset dan melakukan data splitting untuk membagi data menjadi data training dan data validasi. Dataset ini memiliki ukuran piksel 224 x 224 x 3, sehingga



target size disesuaikan dengan ukuran dataset yaitu 224 x 224. Splitting dilakukan dengan kombinasi 80% untuk training dan 20% untuk validasi. Kode penjelasan tersebut akan diimplementasikan menjadi Gambar 0.3.

```
1 data_path = "dataset/"
2 base_dir = os.path.join(data_path)
3
4 IMAGE_SIZE = 224 #model input size
5 BATCH_SIZE = 64
6
7 datagen = tf.keras.preprocessing.image.ImageDataGenerator(
8     rescale = 1./255,
9     validation_split=0.2)
10
11 train_generator = datagen.flow_from_directory(
12     base_dir,
13     target_size=(IMAGE_SIZE, IMAGE_SIZE),
14     batch_size = BATCH_SIZE,
15     subset='training')
16
17 val_generator = datagen.flow_from_directory(
18     base_dir,
19     target_size=(IMAGE_SIZE, IMAGE_SIZE),
20     batch_size = BATCH_SIZE,
21     subset='validation')

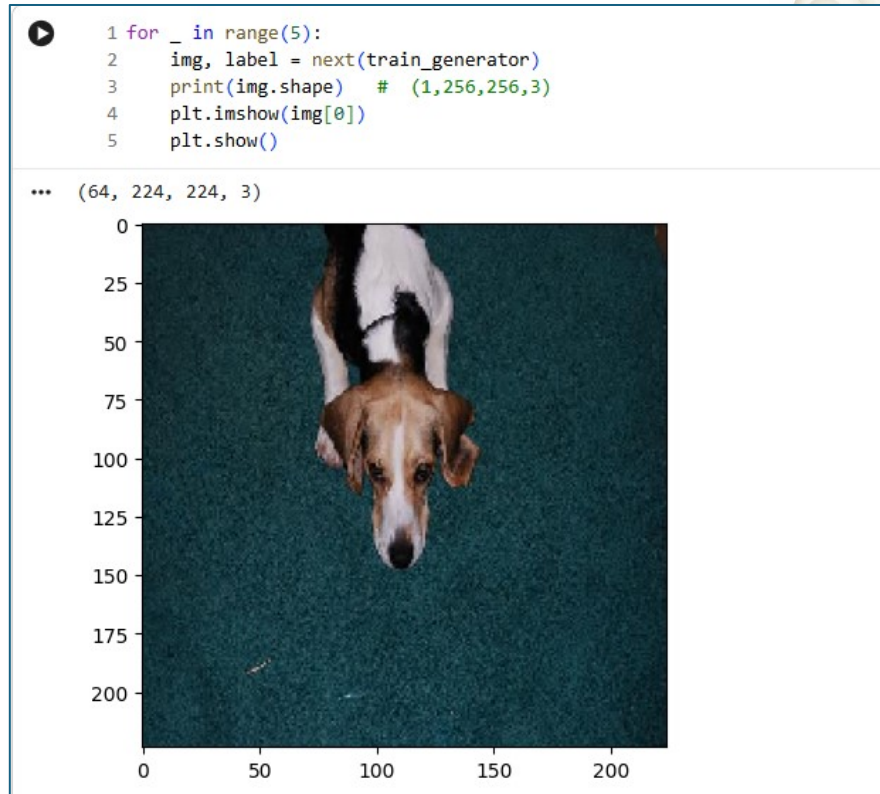
```

... Found 80 images belonging to 2 classes.
Found 20 images belonging to 2 classes.

Gambar 0.3 Data Preparation dan Splitting

4. Uji coba pembacaan data

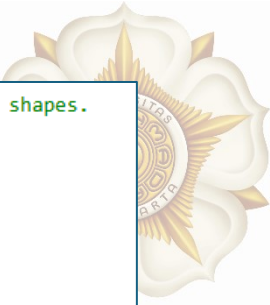
Uji coba pembacaan data guna memastikan bahwa data yang dimuat adalah data yang benar dan sesuai. Jika benar maka akan tertampil seperti Gambar 0.4.



Gambar 0.4 Pengujian pembacaan data

5. Cek kewarasan

Untuk memastikan bahwa data yang dimuat adalah benar, pengecekan dengan membaca langsung data training atau validasi dapat dilakukan. Lakukan pendefinisian ulang terhadap IMG_SHAPE dengan nilai yang dikeluarkan oleh bentuk dari data langsung tersebut. Dengan demikian, kita yakin bahwa informasi data yang akan diberikan ke model CNN adalah benar. Berikut adalah cara mengambil informasi untuk membuat informasi IMG_SHAPE.



```

1 # This loop gets one batch of images and labels to inspect their shapes.
2 # The 'break' ensures it only runs for the first batch.
3 for image_batch, label_batch in train_generator:
4     break
5
6 print(f"Image batch shape: {image_batch.shape}")
7 print(f"Label batch shape: {label_batch.shape}")
8
9 IMG_SHAPE = (IMAGE_SIZE, IMAGE_SIZE, 3)
10
11 num_classes = 2

```

Image batch shape: (16, 224, 224, 3)
Label batch shape: (16, 2)

Gambar 0.5 Pendefinisian IMG_SHAPE dan kelas

6. Model CNN

Buat model CNN dengan Keras API dari tensorflow. Model ini akan membuat 2 lapis blok konvolusi. Seperti yang telah dipelajari di atas pada topik 0, implementasi teori tersebut akan terlihat seperti Gambar 0.6 Model CNN sederhana.

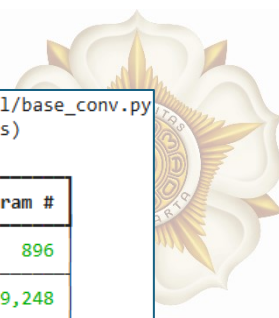
```

1 model = Sequential()
2
3 #-----
4 # Conv Block 1: 32 Filters, MaxPool.
5 #-----
6 model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=IMG_SHAPE))
7 model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
8 model.add(MaxPooling2D(pool_size=(2, 2)))
9 model.add(Dropout(0.1))
10 #-----
11 # Conv Block 2: 64 Filters, MaxPool.
12 #-----
13 model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
14 model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
15 model.add(MaxPooling2D(pool_size=(2, 2)))
16
17 # Flatten the convolutional features.
18 #-----
19 model.add(Flatten())
20 model.add(Dense(512, activation='relu'))
21 model.add(Dense(num_classes, activation='softmax'))
22
23 model.compile(optimizer='rmsprop',
24               loss='categorical_crossentropy',
25               metrics=['accuracy'],
26               )
27
28 model.summary()

```

Gambar 0.6 Model CNN sederhana

Bagian akhir Gambar 0.6 menjalankan fungsi summary. Fungsi tersebut akan menampilkan ringkasan model yang telah dibuat. Hasil ringkasan akan terlihat seperti Gambar 0.6. Model ini memiliki 102,827,554 parameter yang akan dilatih.



```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
    Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	896
conv2d_1 (Conv2D)	(None, 224, 224, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
dropout (Dropout)	(None, 112, 112, 32)	0
conv2d_2 (Conv2D)	(None, 112, 112, 64)	18,496
conv2d_3 (Conv2D)	(None, 112, 112, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
flatten (Flatten)	(None, 200704)	0
dense (Dense)	(None, 512)	102,760,960
dense_1 (Dense)	(None, 2)	1,026

Total params: 102,827,554 (392.26 MB)
 Trainable params: 102,827,554 (392.26 MB)
 Non-trainable params: 0 (0.00 B)

Gambar 0.7 Ringkasan model CNN sederhana

7. Latih model

Latih model CNN dengan data yang telah disiapkan seperti Gambar 0.6. Sebagai uji coba (smoke test) dan mempersingkat waktu, model akan dilatih sebanyak 20 epochs. Simpan hasil evaluasi seperti baris 5 Gambar 0.8.

```

[8] 1 epochs = 20
    2 history = model.fit(train_generator,
    3     epochs=epochs,
    4     validation_data=val_generator)
    5 loss, acc = model.evaluate(val_generator, verbose=1)

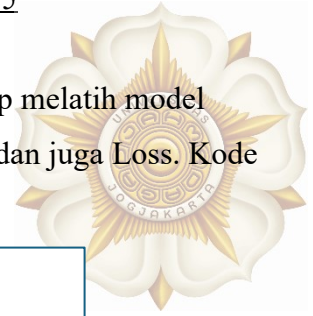
```

... /usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your self._warn_if_super_not_called()

Epoch 1/20	34s 23s/step - accuracy: 0.3875 - loss: 37.3456 - val_accuracy: 0.5000 - val_loss: 1.1778
Epoch 2/20	1s 430ms/step - accuracy: 0.5208 - loss: 1.0483 - val_accuracy: 0.5000 - val_loss: 0.6916
Epoch 3/20	1s 440ms/step - accuracy: 0.5208 - loss: 0.6975 - val_accuracy: 0.5000 - val_loss: 0.6750
Epoch 4/20	1s 159ms/step - accuracy: 0.4896 - loss: 0.7161 - val_accuracy: 0.5000 - val_loss: 0.7077
Epoch 5/20	1s 155ms/step - accuracy: 0.5104 - loss: 0.7154 - val_accuracy: 0.5000 - val_loss: 0.6926
Epoch 6/20	1s 149ms/step - accuracy: 0.5083 - loss: 0.6928 - val_accuracy: 0.5000 - val_loss: 0.6899
Epoch 7/20	1s 154ms/step - accuracy: 0.4698 - loss: 0.6865 - val_accuracy: 0.5000 - val_loss: 0.6866
Epoch 8/20	1s 146ms/step - accuracy: 0.4563 - loss: 0.6927 - val_accuracy: 0.5000 - val_loss: 0.6847
Epoch 9/20	1s 147ms/step - accuracy: 0.5000 - loss: 0.6818 - val_accuracy: 0.6500 - val_loss: 0.6891
Epoch 10/20	1s 425ms/step - accuracy: 0.6958 - loss: 0.6711 - val_accuracy: 0.5000 - val_loss: 0.6758

Gambar 0.8 Latih model CNN sederhana

8. Evaluasi



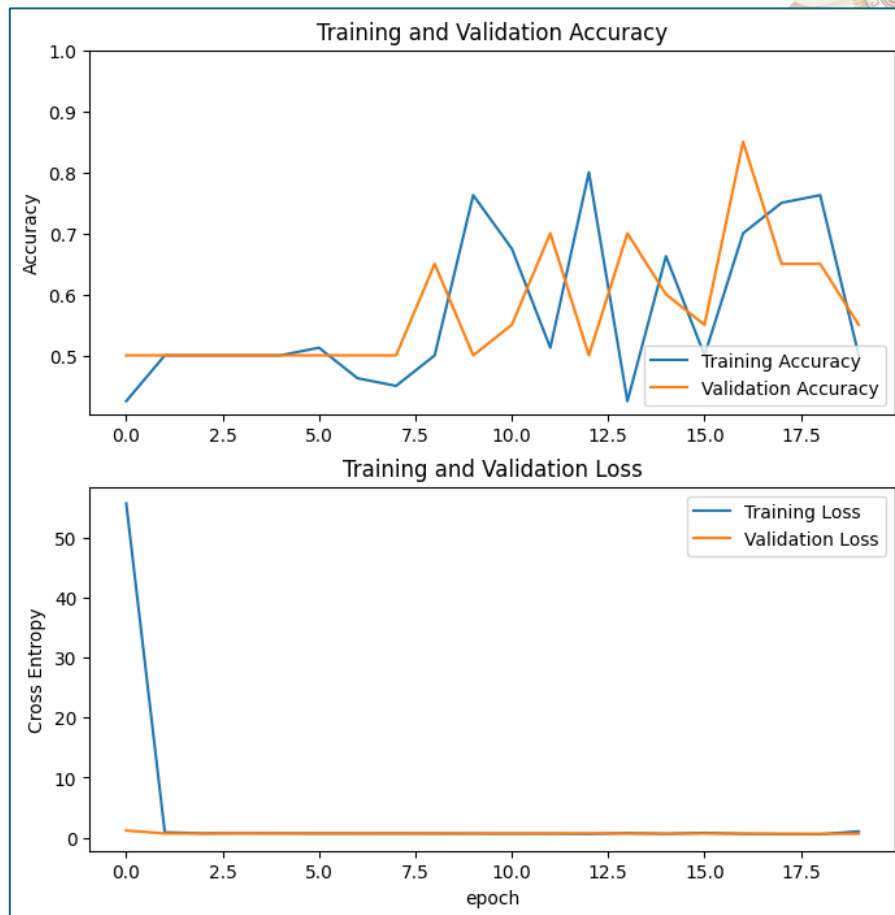
Plot hasil evaluasi yang telah disimpan ke dalam variabel pada tahap melatih model untuk melakukan evaluasi. Evaluasi ini akan menampilkan akurasi dan juga Loss. Kode untuk membuat plotting dapat dilihat di Gambar 0.9.

```
1 acc = history.history['accuracy']
2 val_acc = history.history['val_accuracy']
3 loss = history.history['loss']
4 val_loss = history.history['val_loss']
5
6 plt.figure(figsize=(8, 8))
7 plt.subplot(2, 1, 1)
8 plt.plot(acc, label='Training Accuracy')
9 plt.plot(val_acc, label='Validation Accuracy')
10 plt.legend(loc='lower right')
11 plt.ylabel('Accuracy')
12 plt.ylim([min(plt.ylim()),1])
13 plt.title('Training and Validation Accuracy')
14
15 plt.subplot(2, 1, 2)
16 plt.plot(loss, label='Training Loss')
17 plt.plot(val_loss, label='Validation Loss')
18 plt.legend(loc='upper right')
19 plt.ylabel('Cross Entropy')
20 plt.title('Training and Validation Loss')
21 plt.xlabel('epoch')
22 plt.show()
```

Gambar 0.9 Kode plot evaluasi

Sebagai contoh dari hasil uji coba pelatihan model, didapatkan bahwa model ini tidak mengalami overfitting. Kesimpulan tersebut dapat diambil salah satu caranya dengan cara visual terhadap Gambar 0.10, yaitu akurasi pada tahap validasi tidak tertinggal jauh dari tahap training. Pada kasus ini, validasi justru lebih tinggi dan memiliki nilai

akurasi yang rendah. Hal tersebut mungkin terjadi karena sedikitnya jumlah dataset yang tersedia. Untuk meningkatkan dapat dilakukan penambahan dataset.



Gambar 0.10 Plot evaluasi

9.5. TUGAS & ANALISIS

1. Carilah dataset berisi gambar, boleh berupa klasifikasi binary maupun multi-class! Jumlah dataset tidak harus besar.
2. Bangun sebuah model klasifikasi citra menggunakan CNN untuk melakukan training dan melakukan prediksi pada dataset tersebut!
3. Tampilkan hasil prediksi berupa metrik evaluasi serta visualisasi grafik loss dan akurasi selama proses pelatihan!

9.6. REFERENSI

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.

